

model-tailoring

April 21, 2026

1 Model Tailoring Tutorial

This tutorial will teach you how to customize existing models without the need for the base source code by using the existing pretrained resnet50 in the Pytorch library as a base architecture. This simulates a realistic situation where you only have the model but not the training code and want to build on existing work via transfer learning.

```
[1]: import torch
      from torchvision.models import resnet50, ResNet50_Weights
```

In this tutorial, we will use a tensor of shape [2, 3, 224, 224] (224 x 224 images with a batch size of 2) as an input. As the model is pretrained from the ImageNet dataset, the untailored model should return the output of 1000 class (a tensor of shape [2, 1000]) as an output.

```
[2]: model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V1).cuda()
      inputs = torch.ones((2, 3, 224, 224)).cuda()
```

```
[3]: outputs = model(inputs)
      print(outputs.shape)
```

```
torch.Size([2, 1000])
```

The existing model uses global average pooling and a linear layer after the output from the last convolutional layer to reduce the number of channels to 1000. However, for the sake of this tutorial, we will instead flatten the output from the last convolutional layer and condense it using a linear layer to reduce the number of classes into 10.

As shown below, by printing the model, the global average pooling and a linear layer are in the layers `avgpool` and `fc`, respectively.

```
[4]: print(model)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
    bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1,
    ceil_mode=False)
```

```

(layer1): Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
  (2): Bottleneck(
    (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
)
(layer2): Sequential(
  (0): Bottleneck(

```

```

        (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
        (downsample): Sequential(
          (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
          (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        )
      )
    (1): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (3): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,

```

```

1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
    )
    )
    (layer3): Sequential(
      (0): Bottleneck(
        (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
        (downsample): Sequential(
          (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
          (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        )
      )
      (1): Bottleneck(
        (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
      )
      (2): Bottleneck(
        (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,

```

```

track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
)
(3): Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
)
(4): Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
)
(5): Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
)
)
(layer4): Sequential(
  (0): Bottleneck(

```

```

        (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
        (downsample): Sequential(
          (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
          (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        )
      )
    (1): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=2048, out_features=1000, bias=True)
)

```

To do as mentioned, we change the `fc` and `avgpool` layer to an identity layer instead so that these layers would do nothing.

```
[5]: from typing import cast
import torch.nn as nn

print(model.fc)
model.fc = cast(nn.Linear, torch.nn.Identity(2048))

print(model.avgpool)
model.avgpool = cast(nn.AdaptiveAvgPool2d, torch.nn.Identity(2048))
```

```
Linear(in_features=2048, out_features=1000, bias=True)
```

```
AdaptiveAvgPool2d(output_size=(1, 1))
```

By changing the the `fc` and `avgpool` layer to an identity layer, the model instead return a tensor of size 100352 ([2048, 224//32, 224//32]).

```
[6]: outputs = model(inputs)
print(outputs.shape)
```

```
torch.Size([2, 100352])
```

By applying the `Linear` layer, the classes are now reduced to 10.

```
[7]: new_dense = torch.nn.Linear(100352, 10).cuda()
outputs = new_dense(model(inputs))
print(outputs.shape)
```

```
torch.Size([2, 10])
```

Tips 0: The nested layer, e.g., the last convolution layer, could be accessed using the following command.

```
[8]: model.layer4[2].conv3
```

```
[8]: Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
```

Tips 1: The weight of each layer could be obtained and modified by using `model.named_parameters()`.

```
[9]: for name, param in model.named_parameters():
    print(name, "\t", param.shape, "\t", param.mean())
```

```
conv1.weight      torch.Size([64, 3, 7, 7])      tensor(-0.0005,
device='cuda:0', grad_fn=<MeanBackward0>)
bn1.weight        torch.Size([64])                  tensor(0.2574, device='cuda:0',
grad_fn=<MeanBackward0>)
bn1.bias          torch.Size([64])                  tensor(0.2107, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.0.conv1.weight  torch.Size([64, 64, 1, 1])      tensor(-0.0044,
```

```

device='cuda:0', grad_fn=<MeanBackward0>)
layer1.0.bn1.weight      torch.Size([64])          tensor(0.1879, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.0.bn1.bias        torch.Size([64])          tensor(0.0309, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.0.conv2.weight     torch.Size([64, 64, 3, 3])    tensor(0.0008,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.0.bn2.weight       torch.Size([64])          tensor(0.1595, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.0.bn2.bias        torch.Size([64])          tensor(0.1063, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.0.conv3.weight     torch.Size([256, 64, 1, 1])    tensor(0.0004,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.0.bn3.weight       torch.Size([256])          tensor(0.1268, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.0.bn3.bias        torch.Size([256])          tensor(0.0299, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.0.downsample.0.weight torch.Size([256, 64, 1, 1])    tensor(-0.0034,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.0.downsample.1.weight torch.Size([256])          tensor(0.2101,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.0.downsample.1.bias torch.Size([256])          tensor(0.0299,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.1.conv1.weight     torch.Size([64, 256, 1, 1])    tensor(0.0013,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.1.bn1.weight       torch.Size([64])          tensor(0.1825, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.1.bn1.bias        torch.Size([64])          tensor(-0.0202,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.1.conv2.weight     torch.Size([64, 64, 3, 3])    tensor(4.9314e-05,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.1.bn2.weight       torch.Size([64])          tensor(0.1844, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.1.bn2.bias        torch.Size([64])          tensor(-0.0026,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.1.conv3.weight     torch.Size([256, 64, 1, 1])    tensor(-0.0002,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.1.bn3.weight       torch.Size([256])          tensor(0.0771, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.1.bn3.bias        torch.Size([256])          tensor(0.0008, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.2.conv1.weight     torch.Size([64, 256, 1, 1])    tensor(0.0001,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.2.bn1.weight       torch.Size([64])          tensor(0.1825, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.2.bn1.bias        torch.Size([64])          tensor(-0.0194,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.2.conv2.weight     torch.Size([64, 64, 3, 3])    tensor(-0.0009,

```



```

device='cuda:0', grad_fn=<MeanBackward0>)
layer1.2.bn2.weight      torch.Size([64])      tensor(0.2218, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.2.bn2.bias        torch.Size([64])      tensor(-0.0485,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.2.conv3.weight     torch.Size([256, 64, 1, 1])    tensor(-0.0025,
device='cuda:0', grad_fn=<MeanBackward0>)
layer1.2.bn3.weight       torch.Size([256])      tensor(0.0743, device='cuda:0',
grad_fn=<MeanBackward0>)
layer1.2.bn3.bias         torch.Size([256])      tensor(-0.0115,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.0.conv1.weight     torch.Size([128, 256, 1, 1])    tensor(-0.0015,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.0.bn1.weight       torch.Size([128])      tensor(0.2057, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.0.bn1.bias         torch.Size([128])      tensor(-0.0685,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.0.conv2.weight     torch.Size([128, 128, 3, 3])    tensor(-0.0005,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.0.bn2.weight       torch.Size([128])      tensor(0.2014, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.0.bn2.bias         torch.Size([128])      tensor(0.0186, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.0.conv3.weight     torch.Size([512, 128, 1, 1])    tensor(-0.0001,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.0.bn3.weight       torch.Size([512])      tensor(0.1047, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.0.bn3.bias         torch.Size([512])      tensor(0.0160, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.0.downsample.0.weight torch.Size([512, 256, 1, 1])    tensor(-0.0004,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.0.downsample.1.weight torch.Size([512])      tensor(0.1261,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.0.downsample.1.bias torch.Size([512])      tensor(0.0160,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.1.conv1.weight     torch.Size([128, 512, 1, 1])    tensor(-7.4768e-05,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.1.bn1.weight       torch.Size([128])      tensor(0.1167, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.1.bn1.bias         torch.Size([128])      tensor(0.0425, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.1.conv2.weight     torch.Size([128, 128, 3, 3])    tensor(-3.7090e-05,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.1.bn2.weight       torch.Size([128])      tensor(0.1636, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.1.bn2.bias         torch.Size([128])      tensor(0.0046, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.1.conv3.weight     torch.Size([512, 128, 1, 1])    tensor(-0.0012,

```

```

device='cuda:0', grad_fn=<MeanBackward0>)
layer2.1.bn3.weight      torch.Size([512])      tensor(0.0763, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.1.bn3.bias        torch.Size([512])      tensor(-0.0190,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.2.conv1.weight     torch.Size([128, 512, 1, 1])    tensor(-0.0008,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.2.bn1.weight       torch.Size([128])      tensor(0.1709, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.2.bn1.bias         torch.Size([128])      tensor(0.0033, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.2.conv2.weight     torch.Size([128, 128, 3, 3])    tensor(-0.0003,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.2.bn2.weight       torch.Size([128])      tensor(0.1825, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.2.bn2.bias         torch.Size([128])      tensor(-0.0056,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.2.conv3.weight     torch.Size([512, 128, 1, 1])    tensor(-0.0005,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.2.bn3.weight       torch.Size([512])      tensor(0.1013, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.2.bn3.bias         torch.Size([512])      tensor(-0.0554,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.3.conv1.weight     torch.Size([128, 512, 1, 1])    tensor(-0.0010,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.3.bn1.weight       torch.Size([128])      tensor(0.1702, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.3.bn1.bias         torch.Size([128])      tensor(-0.0271,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.3.conv2.weight     torch.Size([128, 128, 3, 3])    tensor(-0.0007,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.3.bn2.weight       torch.Size([128])      tensor(0.2028, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.3.bn2.bias         torch.Size([128])      tensor(-0.0622,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.3.conv3.weight     torch.Size([512, 128, 1, 1])    tensor(-0.0012,
device='cuda:0', grad_fn=<MeanBackward0>)
layer2.3.bn3.weight       torch.Size([512])      tensor(0.0857, device='cuda:0',
grad_fn=<MeanBackward0>)
layer2.3.bn3.bias         torch.Size([512])      tensor(-0.0537,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.0.conv1.weight     torch.Size([256, 512, 1, 1])    tensor(-0.0013,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.0.bn1.weight       torch.Size([256])      tensor(0.2335, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.0.bn1.bias         torch.Size([256])      tensor(-0.1279,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.0.conv2.weight     torch.Size([256, 256, 3, 3])    tensor(-0.0004,

```

```

device='cuda:0', grad_fn=<MeanBackward0>)
layer3.0.bn2.weight      torch.Size([256])      tensor(0.1817, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.0.bn2.bias        torch.Size([256])      tensor(0.0332, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.0.conv3.weight     torch.Size([1024, 256, 1, 1])  tensor(-0.0004,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.0.bn3.weight       torch.Size([1024])      tensor(0.1378, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.0.bn3.bias         torch.Size([1024])      tensor(-0.0100,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.0.downsample.0.weight torch.Size([1024, 512, 1, 1])
tensor(3.7971e-05, device='cuda:0', grad_fn=<MeanBackward0>)
layer3.0.downsample.1.weight torch.Size([1024])      tensor(0.1094,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.0.downsample.1.bias torch.Size([1024])      tensor(-0.0100,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.1.conv1.weight     torch.Size([256, 1024, 1, 1])  tensor(-0.0005,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.1.bn1.weight       torch.Size([256])      tensor(0.1570, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.1.bn1.bias         torch.Size([256])      tensor(-0.0136,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.1.conv2.weight     torch.Size([256, 256, 3, 3])  tensor(-0.0003,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.1.bn2.weight       torch.Size([256])      tensor(0.1863, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.1.bn2.bias         torch.Size([256])      tensor(-0.0546,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.1.conv3.weight     torch.Size([1024, 256, 1, 1])  tensor(-0.0011,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.1.bn3.weight       torch.Size([1024])      tensor(0.1022, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.1.bn3.bias         torch.Size([1024])      tensor(-0.0535,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.2.conv1.weight     torch.Size([256, 1024, 1, 1])  tensor(-0.0005,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.2.bn1.weight       torch.Size([256])      tensor(0.1554, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.2.bn1.bias         torch.Size([256])      tensor(-0.0460,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.2.conv2.weight     torch.Size([256, 256, 3, 3])  tensor(-0.0007,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.2.bn2.weight       torch.Size([256])      tensor(0.1839, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.2.bn2.bias         torch.Size([256])      tensor(-0.0690,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.2.conv3.weight     torch.Size([1024, 256, 1, 1])  tensor(-0.0008,

```

```

device='cuda:0', grad_fn=<MeanBackward0>)
layer3.2.bn3.weight      torch.Size([1024])      tensor(0.0897, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.2.bn3.bias        torch.Size([1024])      tensor(-0.0579,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.3.conv1.weight     torch.Size([256, 1024, 1, 1])  tensor(-0.0008,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.3.bn1.weight       torch.Size([256])          tensor(0.1647, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.3.bn1.bias         torch.Size([256])          tensor(-0.0783,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.3.conv2.weight     torch.Size([256, 256, 3, 3])  tensor(-0.0008,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.3.bn2.weight       torch.Size([256])          tensor(0.1724, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.3.bn2.bias         torch.Size([256])          tensor(-0.0665,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.3.conv3.weight     torch.Size([1024, 256, 1, 1])  tensor(-0.0012,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.3.bn3.weight       torch.Size([1024])          tensor(0.0880, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.3.bn3.bias         torch.Size([1024])          tensor(-0.0628,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.4.conv1.weight     torch.Size([256, 1024, 1, 1])  tensor(-0.0010,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.4.bn1.weight       torch.Size([256])          tensor(0.1667, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.4.bn1.bias         torch.Size([256])          tensor(-0.0879,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.4.conv2.weight     torch.Size([256, 256, 3, 3])  tensor(-0.0009,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.4.bn2.weight       torch.Size([256])          tensor(0.1776, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.4.bn2.bias         torch.Size([256])          tensor(-0.0883,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.4.conv3.weight     torch.Size([1024, 256, 1, 1])  tensor(-0.0016,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.4.bn3.weight       torch.Size([1024])          tensor(0.0911, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.4.bn3.bias         torch.Size([1024])          tensor(-0.0833,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.5.conv1.weight     torch.Size([256, 1024, 1, 1])  tensor(-0.0010,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.5.bn1.weight       torch.Size([256])          tensor(0.1840, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.5.bn1.bias         torch.Size([256])          tensor(-0.1076,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.5.conv2.weight     torch.Size([256, 256, 3, 3])  tensor(-0.0009,

```

```

device='cuda:0', grad_fn=<MeanBackward0>)
layer3.5.bn2.weight      torch.Size([256])      tensor(0.1932, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.5.bn2.bias        torch.Size([256])      tensor(-0.0891,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.5.conv3.weight     torch.Size([1024, 256, 1, 1])  tensor(-0.0023,
device='cuda:0', grad_fn=<MeanBackward0>)
layer3.5.bn3.weight       torch.Size([1024])      tensor(0.1062, device='cuda:0',
grad_fn=<MeanBackward0>)
layer3.5.bn3.bias        torch.Size([1024])      tensor(-0.1111,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.0.conv1.weight     torch.Size([512, 1024, 1, 1])  tensor(-0.0014,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.0.bn1.weight       torch.Size([512])      tensor(0.2207, device='cuda:0',
grad_fn=<MeanBackward0>)
layer4.0.bn1.bias        torch.Size([512])      tensor(-0.1770,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.0.conv2.weight     torch.Size([512, 512, 3, 3])  tensor(-0.0005,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.0.bn2.weight       torch.Size([512])      tensor(0.1956, device='cuda:0',
grad_fn=<MeanBackward0>)
layer4.0.bn2.bias        torch.Size([512])      tensor(-0.0703,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.0.conv3.weight     torch.Size([2048, 512, 1, 1])  tensor(-0.0006,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.0.bn3.weight       torch.Size([2048])      tensor(0.3619, device='cuda:0',
grad_fn=<MeanBackward0>)
layer4.0.bn3.bias        torch.Size([2048])      tensor(-0.0539,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.0.downsample.0.weight torch.Size([2048, 1024, 1, 1])
tensor(-7.1178e-05, device='cuda:0', grad_fn=<MeanBackward0>)
layer4.0.downsample.1.weight torch.Size([2048])      tensor(0.2683,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.0.downsample.1.bias torch.Size([2048])      tensor(-0.0539,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.1.conv1.weight     torch.Size([512, 2048, 1, 1])  tensor(-0.0009,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.1.bn1.weight       torch.Size([512])      tensor(0.1939, device='cuda:0',
grad_fn=<MeanBackward0>)
layer4.1.bn1.bias        torch.Size([512])      tensor(-0.1236,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.1.conv2.weight     torch.Size([512, 512, 3, 3])  tensor(-0.0008,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.1.bn2.weight       torch.Size([512])      tensor(0.2103, device='cuda:0',
grad_fn=<MeanBackward0>)
layer4.1.bn2.bias        torch.Size([512])      tensor(-0.1203,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.1.conv3.weight     torch.Size([2048, 512, 1, 1])  tensor(-0.0001,

```

```

device='cuda:0', grad_fn=<MeanBackward0>)
layer4.1.bn3.weight      torch.Size([2048])      tensor(0.3577, device='cuda:0',
grad_fn=<MeanBackward0>)
layer4.1.bn3.bias        torch.Size([2048])      tensor(-0.0680,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.2.conv1.weight    torch.Size([512, 2048, 1, 1])  tensor(-0.0005,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.2.bn1.weight      torch.Size([512])      tensor(0.2167, device='cuda:0',
grad_fn=<MeanBackward0>)
layer4.2.bn1.bias        torch.Size([512])      tensor(-0.1719,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.2.conv2.weight    torch.Size([512, 512, 3, 3])  tensor(-0.0007,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.2.bn2.weight      torch.Size([512])      tensor(0.2120, device='cuda:0',
grad_fn=<MeanBackward0>)
layer4.2.bn2.bias        torch.Size([512])      tensor(-0.0877,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.2.conv3.weight    torch.Size([2048, 512, 1, 1])  tensor(-3.3771e-05,
device='cuda:0', grad_fn=<MeanBackward0>)
layer4.2.bn3.weight      torch.Size([2048])      tensor(0.7160, device='cuda:0',
grad_fn=<MeanBackward0>)
layer4.2.bn3.bias        torch.Size([2048])      tensor(0.0246, device='cuda:0',
grad_fn=<MeanBackward0>)

```

Tips 2: In case that you want you modify multiple layers programatically, you could access each layer using `model.modules()`.

```

[10]: for module in model.modules():
        print(module.__class__.__name__, ": ", module)
        print("-----")

```

```

ResNet : ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1,
ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)

```

```

        (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
        (downsample): Sequential(
          (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        )
      )
    (1): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (layer2): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,

```

```

track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
  (2): Bottleneck(
    (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
  (3): Bottleneck(
    (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
)
)

```



```

(layer3): Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
  (2): Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
  (3): Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,

```

```

track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
)
(4): Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
)
(5): Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
)
)
(layer4): Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,

```

```

track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
  (2): Bottleneck(
    (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
    (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
  )
  (avgpool): Identity()
  (fc): Identity()
)
----
Conv2d : Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False)
----
BatchNorm2d : BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
MaxPool2d : MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1,

```

```

ceil_mode=False)
----
Sequential : Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
  (2): Bottleneck(
    (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
)
)

```

```

----
Bottleneck : Bottleneck(
  (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
  (downsample): Sequential(
    (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
)
----
Conv2d : Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
Sequential : Sequential(
  (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
)
----
Conv2d : Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)

```

```

----
Bottleneck : Bottleneck(
  (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
----
Conv2d : Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
Bottleneck : Bottleneck(
  (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
----
Conv2d : Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)

```

```

----
BatchNorm2d : BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
Sequential : Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (1): Bottleneck(
    (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)

```

```

)
(2): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
(3): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
)
----
Bottleneck : Bottleneck(
  (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
  (downsample): Sequential(
    (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
    (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
)
)
----

```



```

Conv2d : Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
Sequential : Sequential(
  (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
  (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
)
----
Conv2d : Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
----
BatchNorm2d : BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Bottleneck : Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
----
Conv2d : Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----

```

```

Conv2d : Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
Bottleneck : Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
----
Conv2d : Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
Bottleneck : Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,

```

```

track_running_stats=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
)
----
Conv2d : Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
Sequential : Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
)

```

```

(1): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
(2): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
(3): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
(4): Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)

```

```

        (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
    )
    (5): Bottleneck(
        (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
    )
)
----
Bottleneck : Bottleneck(
    (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
        (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
)
----
Conv2d : Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)

```

```

----
Conv2d : Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
Sequential : Sequential(
  (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
  (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
)
----
Conv2d : Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
----
BatchNorm2d : BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Bottleneck : Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
----
Conv2d : Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)

```

```

----
ReLU : ReLU(inplace=True)
----

Bottleneck : Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
)
----

Conv2d : Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
----

BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----

Conv2d : Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----

BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----

Conv2d : Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
----

BatchNorm2d : BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----

ReLU : ReLU(inplace=True)
----

Bottleneck : Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
)

```

```

----
Conv2d : Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
Bottleneck : Bottleneck(
  (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
----
Conv2d : Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)

```



```

----
ReLU : ReLU(inplace=True)
----

Bottleneck : Bottleneck(
    (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
)
----

Conv2d : Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
----

BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----

Conv2d : Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----

BatchNorm2d : BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----

Conv2d : Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
----

BatchNorm2d : BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----

ReLU : ReLU(inplace=True)
----

Sequential : Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
    (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (relu): ReLU(inplace=True)
  )
)

```

```

        (downsample): Sequential(
          (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
          (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        )
      )
      (1): Bottleneck(
        (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
      )
      (2): Bottleneck(
        (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (relu): ReLU(inplace=True)
      )
    )
  )
  ----
Bottleneck : Bottleneck(
  (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
  (downsample): Sequential(
    (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)

```

```

        (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
)
----
Conv2d : Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
Sequential : Sequential(
  (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
  (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
)
----
Conv2d : Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
----
BatchNorm2d : BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Bottleneck : Bottleneck(
  (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
----

```

```

Conv2d : Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
ReLU : ReLU(inplace=True)
----
Bottleneck : Bottleneck(
  (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
  (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (relu): ReLU(inplace=True)
)
----
Conv2d : Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
----
BatchNorm2d : BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----
Conv2d : Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
----
BatchNorm2d : BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
----

```

```
ReLU : ReLU(inplace=True)
----
Identity : Identity()
----
Identity : Identity()
----
```

Tips 3: Actually, we can change the number of class of the pretrained ResNet 50 by using the input argument `num_classes`. But this is not as cool.

```
[11]: model = resnet50(num_classes=3).cuda()
      model(inputs).shape
```

```
[11]: torch.Size([2, 3])
```

1.1 More Convoluted Modification

The methods above could also perform simple operations like altering and removing layers, however, it could not perform more complex operations like forking. In this part, we will show how to perform a more complex operation by modifying the model to return multi-scale predictions by introducing an additional head that performs global average pooling followed by a linear layer after the last convolutional layer of the fourth stage as shown in Figure below.

To do so, we will use the operation `register_forward_hook` to get the intermediate output of the last convolutional layer of stage 4.

```
[12]: from torch import relu

class MultiscaleModel(torch.nn.Module):
    def __init__(self):
        super(MultiscaleModel, self).__init__()
        # base_model is a standard imagenet pretrained model
        self.base_model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V1)
        self.activation = {}

        # layer for new prediction head
        self.global_avg_pool = torch.nn.AdaptiveAvgPool2d((1,1)) # Global
        ↪average pooling layer
        self.fc = torch.nn.Linear(1024, 1000)

    def get_activation(name):
        def hook(model, input, output):
            self.activation[name] = output#.detach() # in case that you
            ↪want to stop the gradient to flow through the hook you can detach()
```

```

        return hook

        # base_model.layer3[-1].relu is the last convolutional output of the
        ↪stage 4
        relu: nn.Module = cast(nn.Module, self.base_model.layer3[-1].relu)
        relu.register_forward_hook(get_activation("stage4"))

        def forward(self, inp):
            outputs = self.base_model(inp)
            stage_4_output = self.activation["stage4"]
            print("stage 4 output : ", stage_4_output.shape)

            fourth_stage_head = self.global_avg_pool(stage_4_output)
            fourth_stage_head = fourth_stage_head.view(fourth_stage_head.shape[0],
            ↪fourth_stage_head.shape[1]) # reshape from torch.Size([2, 1024, 1, 1]) to
            ↪torch.Size([2, 1024])
            fourth_stage_head_output = self.fc(fourth_stage_head)

            return outputs, fourth_stage_head_output

```

```

[13]: inputs = torch.ones((2, 3, 224, 224)).cuda()
      model = MultiscaleModel().cuda()

```

The model now returns two outputs : the one after stage 5 (last convolutional layer) and stage 4.

```

[14]: output = model(inputs)
      print(output[0].shape, output[1].shape)

```

```

stage 4 output : torch.Size([2, 1024, 14, 14])
torch.Size([2, 1000]) torch.Size([2, 1000])

```
